

# ssh access

How people reach SD on this machine, why it is ssh, and the one thing it costs you.

**Accounts SD creates cannot log in to Windows at this machine.** They are denied the physical console and Remote Desktop, deliberately and by group membership. Local terminal access belongs to administrators, who have ordinary Windows accounts.

**They reach SD over ssh, or through an API client, or both.** Which of those an account may use is chosen when it is created and is a required keyword — see Per-account control below. This page covers the ssh route; API access covers the other.

This has no equivalent in OpenQM or in SD on Linux, and it shapes almost everything else about running SD Core on Windows.

## The rule, and the one exception

|                                | CREATE.ACCOUNT USER X | CREATE.ACCOUNT USER X ADMINISTRATOR |
|--------------------------------|-----------------------|-------------------------------------|
| Windows group                  | standard user         | Administrators                      |
| Administers SD                 | no                    | yes                                 |
| Local console / Remote Desktop | denied                | allowed                             |
| ssh                            | if granted            | yes                                 |
| API                            | if granted            | yes                                 |

**"Denied the console" is about logging in to Windows, not about reaching sd.** An account with `api` and no `ssh` is a perfectly normal thing to create — a person using a custom GUI client that talks to SD does not need a terminal session at all.

**Multi-user access over remote Desktop is not supported.** This is not an oversight and not a gap to be filled later. One Windows setting covers Remote Desktop and the physical keyboard together, so an account allowed to use RDP could also walk up to the machine and log in — which was never the intention. A verb that lifted the restriction (`RDPACCOUNT`) was built and deleted the next day for exactly that reason.

**The rule that now holds without exception:** nobody SD creates can log in to Windows at this machine unless they are already a Windows administrator. **SD accounts reach the machine over ssh or through an API client.**

If you want multi-user Remote Desktop, you want Windows Server and RDP client access licences, and probably a commercial product built for it.

---

## How it is done

Two Windows user rights, applied **to a group, once** — not per account:

---

|                                   |                             |
|-----------------------------------|-----------------------------|
| SeDenyInteractiveLogonRight       | blocks the physical console |
| SeDenyRemoteInteractiveLogonRight | blocks Remote Desktop       |

---

Both are on the group `sdsshonly`, and `create.account` joins every non-administrator account to it.

**Network logon is not denied, and must not be.** Win32-OpenSSH authenticates with a network logon — cleartext network logon for passwords, S4U for public keys — so denying it would lock out the very access this exists to preserve. **This is the trap in the design and it is the one thing to get right.**

**It cannot be `sdusers`.** That group grants access to the data files and administrators are in it too, so denying console logon there would lock administrators out of their own console. The two groups answer different questions and must stay separate.

---

## An ssh session lands inside SD

`sshd_config` carries a global `ForceCommand`, so signing in over ssh puts you straight into SD rather than at a Windows prompt.

**That applies to everyone who may connect, administrators included** — the rule is about the route in, not about who took it.

THE COST: SCP AND SFTP STOP WORKING INBOUND

**NO FILE CAN BE PUSHED TO THIS MACHINE OVER ssh, BY ANYBODY, ADMINISTRATORS INCLUDED.** The command is forced, so there is no file-transfer subsystem left to run. This is deliberate and is the accepted cost of the above.

**The cost is INBOUND ONLY, and that is the whole of the answer.** `ForceCommand` applies to sessions where this machine is the ssh *server*. WinSCP or `scp` running **on** this machine, connecting outward, makes it the *client*, and `sshd_config` is not consulted at all.

**So copy files by Pulling them, not pushing them.** Sit at the machine or connect with Remote Desktop — both untouched, because administrators are never put in `sdsshonly` — and fetch what you need from there. Outbound connections are not firewalled.

Remote-control tools that copy files are unaffected, and so are the console and Remote Desktop.

**Do not "fix" this with a `Match Group administrators` exemption.** Beyond giving administrators a PowerShell prompt instead of SD — which is *more* access than the global form gives them — it may not even work: `sshd_config` takes the **first** obtained value for a keyword, and SD's block is inserted before the first `Match`, so a later `ForceCommand none` is not guaranteed to override the earlier global one. Pulling avoids the question entirely.

## Who may ssh in at all

`AllowGroups` in `sshd_config` is the second layer: the deny rights stop local logon, `AllowGroups` decides who may connect.

**THE "Limit ssh" CHECKBOX IS GONE. IT IS NOW A STATEMENT.** SD limits ssh to SD users and administrators and puts every ssh session straight into SD. It had been ticked by default for some time and was not really meant to be turned off, so presenting it as a checkbox suggested a choice that was not one. It is described on the *Before you install* page instead, with the scp cost stated plainly.

The reason it can no longer be declined is that the machine it existed for — one with somebody else's ssh server — is now turned away before the install starts.

**Your existing `sshd_config` is kept as `sshd_config.before-sd`, and uninstalling SD puts it back.**

### IF SD DID NOT INSTALL YOUR SSH SERVER

SD never reconfigures or restarts an ssh server it did not install — it may be managed by policy. On such a machine, apply the confinement yourself, once, from an elevated prompt:

```
powershell -File "C:\Program Files\SD\allow-ssh-groups.ps1" -Installed
```

It checks the result with `sshd -T` and puts the original back if anything is wrong, keeping a copy as `sshd_config.before-sd`.

It also **refuses outright** if `sshd_config` already carries an `AllowGroups`, `AllowUsers`, `DenyGroups` or `DenyUsers` line — that is somebody's policy and it is left alone.

## Reaching the machine from the network

**Port 22 is closed to other computers unless you tick the box during installation.** The rule is scoped to `127.0.0.1,::1` otherwise.

**IF YOU INSTALLED A BUILD BEFORE 25 Aug 2026, CHECK THIS.** The narrowing step failed **every single time it ran**, leaving port 22 open to the local network even when the box was unticked. The installer's closing page said so: *"Setting who may reach ssh FAILED, and Windows' own default is in force — port 22 open to your local network."* The recovery command it offered failed for the same reason. Only machines where SD installed ssh itself are affected. Fix it with:

```
powershell -File "C:\Program Files\SD\ssh-firewall.ps1" -Installed -Restrict
```

It now reports *"ssh is reachable FROM THIS MACHINE ONLY"*. Check the current setting with `-Show`, or look at the rule **OpenSSH SSH Server (sshd)** in `wf.msc`.

**Connecting to SD on your own machine is unaffected**, over IPv4 and IPv6 alike — Windows does not filter traffic that never leaves the machine, so `ssh localhost` keeps working whichever way the setting is left. **A local-only installation is served entirely by `ssh localhost`.**

## Per-account control

An account is told at creation which routes it may use, and one of the four keywords is required:

### REACHES SD BY

`create.account user fred ssh`

a terminal session over ssh

`create.account user fred api`

**an API client only** — no terminal, and nothing on this page applies to them

`create.account user fred both`

either

`create.account user fred none`

neither — reachable only with `logto` from another session

An `api` account never touches any of this. No ssh session, no `ForceCommand`, no port 22. For people running a GUI client against SD, that is the account shape they want, and the ssh configuration on this page does not apply to them.

`modify.account fred both` changes it afterwards — and remember the keyword says what the access **is**, not what to add, so `modify.account fred api` takes ssh away. **Administrators always have both, and it cannot be taken from them — but for an administrator both mean from this machine only.** See Account types, and the section on remote administration below.

**A suspended account is refused after the connection is made, not before.** `modify.account fred suspended` does not touch the `sdssh` group, so sshd still accepts Fred's connection and SD still starts — and then refuses him:

Account FRED is suspended

**A suspension is an SD control, not a network one.** That is what makes lifting it free: nothing was withdrawn, so nothing has to be restored. If you need the connection itself refused, take the account out of `sdssh` too with `modify.account fred none` — and remember you must then put it back by hand.

---

## What ssh-only does not mean

**The deny rights control *where* an account may log in, not *what it may run*.** Confining a user to SD rather than to a shell is the separate `ForceCommand` control above, and reaching the operating system from inside SD is the separate `os.users` permit list — see Administrator commands.

**And ssh-only does not give users isolation from each other's data.** Every SD process opens the database under the invoking user's own token, so everyone who uses SD needs file access to the tree and can read another account's directory from outside SD. See Security.

---

## An administrator cannot sign in from another machine

**If your SD account is an ADMINISTRATOR account, SD refuses an ssh connection from another computer.** You will see:

```
An administrator may not sign in to this machine from another one.
```

and the connection ends. The same applies to the API.

**ssh and the API still work for an administrator on this machine.** A loopback connection — `ssh you@localhost`, or an API client running on the same box — is admitted normally. It is *remote* that is refused, not ssh.

**Nothing changes for ordinary accounts.** Standard and programmer accounts keep ssh from anywhere, exactly as before, and so does a Windows administrator whose SD account is an ordinary one. **It is the SD account tier that decides, not Windows group membership.**

### WHERE ADMINISTRATION HAPPENS

- at this machine's own console, or
- through a remote desktop or remote-control product **installed as a service**.

A per-user install of a remote-control tool is not enough: it cannot display the Windows consent prompt, and the operator sees a frozen screen instead.

### WHY

Administration needs a screen Windows can draw its consent prompt on. That prompt is what makes an elevation something a person agreed to, rather than something that merely happened. A connection from another machine has no such screen.

**And the token an ssh session carries is not a filtered one.** OpenSSH runs as a system service and builds the sign-in token itself, so the filtering that applies to an ordinary local sign-in never applies to it. Without this refusal a Windows administrator arriving over ssh would hold **full** administrator rights, with nobody asked to consent to anything. That is the hole this refusal closes.

#### IF YOU RELY ON REMOTE ADMINISTRATION TODAY

**It will stop working when you upgrade, and that is deliberate.** Use the console, or a service-installed remote desktop. If the account only needs ordinary, non-administrative work from another machine, ask an administrator to change its tier instead.

**Scheduled tasks are affected too.** A task that runs unattended has no screen either, so one signing in as an administrator account is refused. Give it an ordinary account and list the command it runs in the SD system file

`batch.jobs`.